

並列分散処理 (ネットワークシステム) 第4章 システム化技術

山内 利宏

岡山大学 工学部 工学科
情報・電気・数理データサイエンス系
情報工学コース

トランザクション管理技術

- トランザクションとは、関連する複数の処理を一つにまとめ、まとめた処理が「すべて成功」または「すべて失敗」のどちらかの結果しか持たないことを保証する処理単位
 - 例えば、銀行業務システムの振り替え
 - ◆ 口座からの「引き落とし」と「振り込み」からなる
 - ◆ 引き落としと振り込みの両方が完了、またはまったく実行しないかのどちらかにする必要がある
- 4つの性質を満たす必要あり

原子性と分離性

- 原子性 (Atomicity)
 - 完了しているか, 完了していないかのどちらかという性質
 - 部分完了のような中途半端な状態は持たない
- 分離性 (Isolation)
 - トランザクションが共用データに与えた影響は, トランザクション完遂後, 他のトランザクションから見える
 - トランザクションに必要な情報はロックされ, 他のトランザクションから変更できない

恒久性と並行性

- 恒久性 (Durability)
 - データへの変更は、システムに障害が生じてでも失われることがない
- 並行性 (Serializability)
 - 複数のトランザクションが並行して動作されても、その結果は任意の順序で実行された結果と等しくなければならない
- 4つの性質を有するトランザクションをアトミックトランザクションと呼ぶ

トランザクションサービス

- ファイルの一部,あるいは全部のファイル操作に対して,アトミックトランザクションを提供するファイルシステム
 - OpenTransactionとCloseTransactionに囲まれたファイル操作列で定義
 - 全操作が正常終了時,そのトランザクションはコミットされたことをクライアントに伝える
 - ◆ コミットされると,トランザクションで要求されたファイルの更新がすべて有効(恒久的)になる
- トランザクションは,強制終了させられることがある
 - これを[アボート](#)と呼ぶ
 - 要因: 他のトランザクションとの競合,ユーザからの要求,プロセスや計算機の障害など
 - アボートが起こった場合,ファイルの状態をトランザクションの開始前に戻す([ロールバック](#)と呼ぶ)

2相コミットメント制御

- コミットやアボートを完全に終了させるための制御
- 2相から構成される
 - 第1相では, サーバは全クライアントにコミットの用意があるか問い合わせる
 - 各クライアントは, 返答するとともに, アボートの権利を放棄し, コミットとアボートができる中間状態に移行
 - サーバは全クライアントからの返答を受け, すべて肯定応答ならコミット, そうでなければアボートを全クライアントに指示する
- これにより, 全クライアントで同一の決定を下すことができる

2相コミットメント制御の図

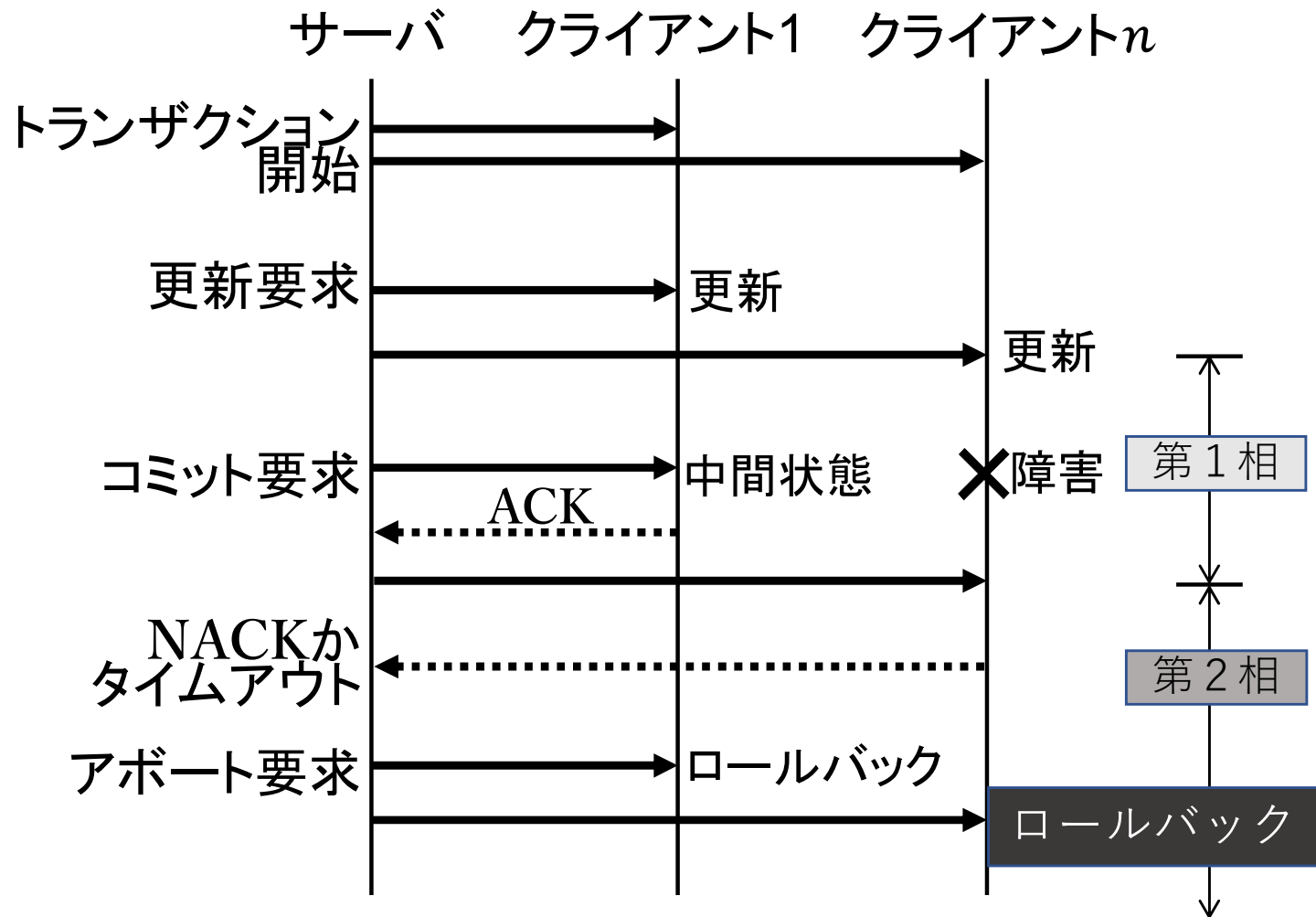
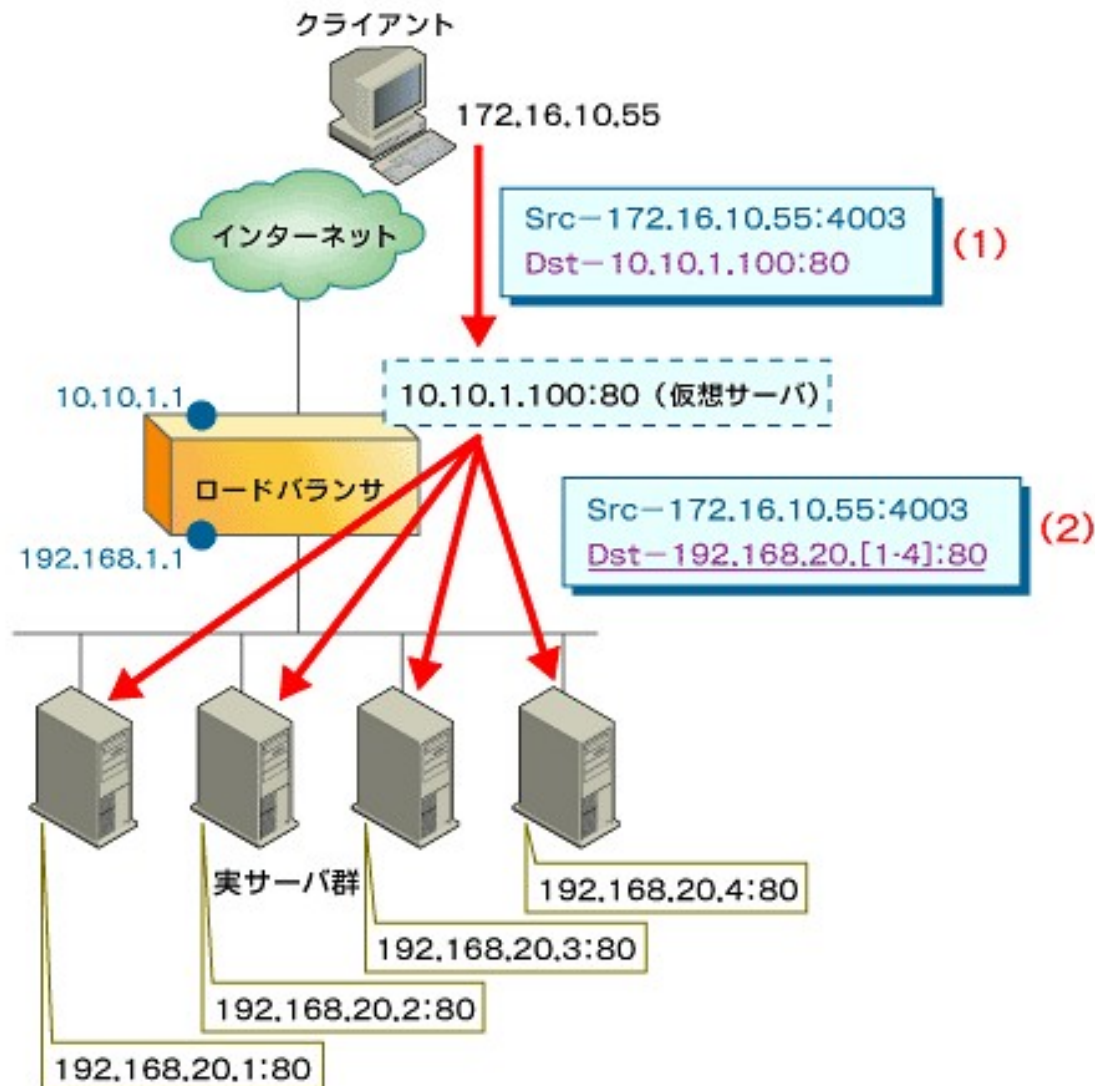


図4.11 2相コミットメント制御

負荷分散

- 分散システムにおける性能向上の手段
 - 負荷をノード間に分散
 - システム全体のスループットや応答時間を向上
- 手段
 - 負荷に応じてプロセスを移送
 - 他に, リクエストを分散させる方式もある

サーバの負荷分散の例



出典: <https://www.atmarkit.co.jp/fnetwork/rensai/lb01/lb01.html>

2025年度並列分散処理

プロセス移送とは

- 実行中のプロセスを他のノードに移動（マイグレーション, Migration）させること
 - 実行環境を移送するため、コストがかかる
 - ◆ プロセスが利用するメモリの内容
 - テキスト部, データ部, スタック部
 - レジスタなどの情報
 - プロセスが利用する資源（データなど）
 - 通信状態も引き継ぐ必要
- 現在は、プロセスの移送よりも、コンテナや仮想マシンの移送が行われることが多い
 - ライブマイグレーション
 - ◆ 例：仮想マシンをできるだけ止めずに移動

Linuxにおけるマイグレーション技術

- Checkpoint/Restore技術
 - CRIU (Checkpoint/Restore In Userspace)
 - ◆ ユーザ空間で動作し, プロセスの状態をファイルに保存して別のマシンで復元可能
- コンテナのライブマイグレーション
 - Docker + CRIU
 - ◆ DockerはCRIUを利用してコンテナの状態を保存し, 別のホストで再開
- 仮想マシンのライブマイグレーション
 - KVM/QEMU + libvirt
 - ◆ 仮想マシンのメモリやCPU状態を転送し、ほぼ停止時間なしで別ホストへ移動

移送要求

- 負荷の重いノードから負荷の軽いノードにプロセスを移送させる方法（送り側主導型）
 - 過負荷時に迅速にノードを解放可能
- 負荷の軽いノードから負荷の重いノードにプロセス移送を要求する方法（受け側主導型）
 - システム全体の効率や稼働率向上

プロセス選択とタイミング

- 実行時間の長くかかりそうなプロセスを選ぶ方法
 - 移送コストに見合うかどうか
 - 実行された時間が長いプロセスは, これからも長く実行されることが予測可能(超指数分布を仮定)

移送コスト

- プログラムの大きさに比例
- しかし、ネットワークトラフィックやノード負荷により、移送コストは変動
 - 統計的処理で影響を予測する必要あり

資源管理技術

- 負荷分散を動的に行うためには、システム全体の資源管理が必要
 - 集中管理方式
 - ◆ 特定のノードが管理
 - ◆ 一貫した資源管理がしやすい
 - ◆ オーバヘッドが小さい
 - 分散管理方式
 - ◆ 各ノード間の協調動作で管理
 - ◆ 複数ノードに分散(集中管理の欠点を解消)
 - ◆ オーバヘッド大
 - 一貫性制御, 通信コスト
 - ◆ 階層的にグループ化して資源管理

管理情報(1/2)

■ 管理情報

- CPU使用状況
 - ◆ プロセスの数や残余処理時間の総和など
- メモリ使用状況
 - ◆ ページフォールトの頻度など
- ディスク使用状況
 - ◆ ディスク利用率や空き領域など
- デバイス使用状況
 - ◆ 接続されている他のデバイスの利用率など

管理情報(2/2)

- ノード間通信量
 - ◆ ノードの入出パケット数など
- ノードの稼働状況
 - ◆ 立ち上がっているかどうか
- ノードの接続状況
 - ◆ 他のノードとのコネクション数と経過時間など
- ノードの特性
 - ◆ ...